УНИВЕРЗИТЕТ У НОВОМ САДУ
ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
У НОВОМ САДУ

Марко Хорстман

Системско пројектовање и хардверска имплементација акцелератора за проналажење путање заснованог на *GBFS* алгоритму

ДИПЛОМСКИ РАД

- Основне академске студије -

Нови Сад, 2026.

	УНИВЕРЗИТЕТ У НОВОМ САДУ ● **ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА** 21000 НОВИ САД, Трг Доситеја Обрадовића 6	Број:
	ЗАДАТАК ЗА ЗАВРШНИ РАД	Датум: 30.9.2026

(Податке уноси предметни наставник - ментор)

Студијски програм:	Енергетика, електроника и телекомуникације		
Студент:	Марко Хорстман	Број индекса:	ЕЕ27/2022
Степен и врста студија:	Основне академске студије		
Област:	Електротехника и рачунарство		
Ментор:	др Вук Врањковић, ванредни професор		

НА ОСНОВУ ПОДНЕТЕ ПРИЈАВЕ, ПРИЛОЖЕНЕ ДОКУМЕНТАЦИЈЕ И ОДРЕДБИ СТАТУТА ФАКУЛТЕТА ИЗДАЈЕ СЕ ЗАДАТАК ЗА ЗАВРШНИ РАД, СА СЛЕДЕЋИМ ЕЛЕМЕНТИМА:

- проблем – тема рада;
- начин решавања проблема и начин практичне провере резултата рада, ако је таква провера неопходна;

НАСЛОВ ЗАВРШНОГ РАДА:

Системско пројектовање и хардверска имплементација акцелератора за проналажење путање заснованог на *GBFS* алгоритму

ТЕКСТ ЗАДАТКА:

Циљ рада је развој и евалуација хардверског акцелератора за проналажење путање на дводимензионалној мапи применом *GBFS* алгоритма. Потребно је развити софтверску имплементацију, профилисањем идентификовати функције погодне за хардверско убрзавање и извршити хардверско-софтверско партиционисање система. Предложену архитектуру моделовати *SystemC/TLM* виртуелном платформом ради провере организације и процене перформанси. Акцелератор пројектовати у *VHDL*-у, уз оптимизацију меморијске организације и паралелизацију претраге, и интегрисати га са *Zynq* процесорским системом преко *AXI4-Lite* интерфејса на *Zybo* развојној плочи. Функционалност и перформансе проверити симулацијом и експерименталним испитивањем, а процењене и остварене резултате упоредити и анализирати.

Руководилац студијског програма:	Ментор рада:
др Милан Сечујски, редовни професор	др Вук Врањковић, ванредни професор

Примерак за: o - Студента; o - Ментора

УНИВЕРЗИТЕТ У НОВОМ САДУ ● **ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА**
21000 НОВИ САД, Трг Доситеја Обрадовића 6

КЉУЧНА ДОКУМЕНТАЦИЈСКА ИНФОРМАЦИЈА

Редни број, **РБР**:			
Идентификациони број, **ИБР**:			
Тип документације, **ТД**:		Монографска документација	
Тип записа, **ТЗ**:		Текстуални штампани материјал	
Врста рада, **ВР**:		Дипломски рад	
Аутор, **АУ**:		Марко Хорстман	
Ментор, **МН**:		др Вук Врањковић, ванредни професор	
Наслов рада, **НР**:		Системско пројектовање и хардверска имплементација акцелератора за проналажење путање заснованог на GBFS алгоритму	
Језик публикације, **ЈП**:		Српски	
Језик извода, **ЈИ**:		Српски / Енглески	
Земља публиковања, **ЗП**:		Србија	
Уже географско подручје, **УГП**:		Војводина	
Година, **ГО**:		2026	
Издавач, **ИЗ**:		Ауторски репринт	
Место и адреса, **МА**:		21000 Нови Сад, Трг Доситеја Обрадовића 6	
Физички опис рада, **ФО**: (поглавља/страна/ цитата/табела/слика/графика/прилога)		9/39/7/10/10/0/0	
Научна област, **НО**:		Електротехника и рачунарство	
Научна дисциплина, **НД**:		Електроника	
Предметна одредница/Кључне речи, **ПО**:		Хардверски акцелератор, *GBFS*, *SystemC*	
УДК			
Чува се, **ЧУ**:		Библиотека Факултета техничких наука 21000 Нови Сад, Трг Доситеја Обрадовића 6	
Важна напомена, **ВН**:		Нема	
Извод, **ИЗ**:		У раду је приказано системско пројектовање и хардверска имплементација акцелератора за проналажење путање на мапи димензија 100×100 чворова, заснованог на *GBFS* алгоритму. На основу профајлирања софтверске спецификације извршено је хардверско-софтверско партиционисање и израђена виртуелна платформа у *SystemC*-у. Акцелератор је описан у *VHDL*-у, са четвороструко паралелизованом претрагом мапе, и интегрисан са *Zynq* процесорским системом преко *AXI4-Lite* интерфејса на *Zybo* плочи. Остварене перформансе и искоришћење ресурса упоређени су са проценама са системског нивоа.	
Датум прихватања теме, **ДП**:		17.06.2026.	
Датум одбране, **ДО**:		16.10.2026.	
Чланови комисије, **КО**:	Председник:	др Иван Мезеи, редовни професор	
	Члан:	др Предраг Теодоровић, ванредни професор	Потпис ментора
	Члан, ментор:	др Вук Врањковић, ванредни професор	

UNIVERSITY OF NOVI SAD ● **FACULTY OF TECHNICAL SCIENCES**
21000 NOVI SAD, Trg Dositeja Obradovića 6

KEY WORDS DOCUMENTATION

Accession number, **ANO**:			
Identification number, **INO**:			
Document type, **DT**:		Monographic publication	
Type of record, **TR**:		Textual material	
Contents code, **CC**:		BSc thesis	
Author, **AU**:		Marko Horstmann	
Mentor, **MN**:		Vuk Vranjković, PhD, Associate Professor	
Title, **TI**:		System-Level Design and Hardware Implementation of a Pathfinding Accelerator Based on the GBFS Algorithm	
Language of text, **LT**:		Serbian	
Language of abstract, **LA**:		Serbian / English	
Country of publication, **CP**:		Serbia	
Locality of publication, **LP**:		Vojvodina	
Publication year, **PY**:		2026	
Publisher, **PB**:		Author's reprint	
Publication place, **PP**:		Novi Sad, Faculty of Technical Sciences, Trg Dositeja Obradovića 6	
Physical description, **PD**: (chapters/pages/ref./tables/pictures/graphs/appendixes)		9/39/7/10/10/0/0	
Scientific field, **SF**:		Electrical Engineering and Computer Science	
Scientific discipline, **SD**:		Electronics	
Subject/Key words, **S/KW**:		Hardware Accelerator, *GBFS*, *SystemC*	
UC			
Holding data, **HD**:		Library of the Faculty of Technical Sciences, Trg Dositeja Obradovića 6, Novi Sad	
Note, **N**:		None	
Abstract, **AB**:		This thesis presents the system-level design and hardware implementation of a pathfinding accelerator for a 100×100 node map, based on the GBFS algorithm. Based on profiling of the software specification, hardware-software partitioning was performed and a virtual platform was developed in SystemC. The accelerator is described in VHDL, with a fourfold parallelized map search, and integrated with the Zynq processing system through an AXI4-Lite interface on the Zybo board. The achieved performance and resource utilization are compared with the system-level estimates.	
Accepted by the Scientific Board on, **ASB**:		17.06.2026.	
Defended on, **DE**:		16.10.2026.	
Defended Board, **DB**:	President:	Ivan Mezei, PhD, Full Professor	
	Member:	Predrag Teodorović, PhD, Associate Professor	Mentor's sign
	Member, Mentor:	Vuk Vranjković, PhD, Associate Professor	

Изјава о академској честитости

Студент : **Марко Хорстман**

Број индекса : **ЕЕ27/2022**

Студент **основних академских студија**

Аутор рада под називом **: Системско пројектовање и хардверска имплементација акцелератора за проналажење путање заснованог на *GBFS* алгоритму**

Потписивањем изјављујем :

- да је рад искључиво резултат мог сопственог истраживачког рада;
- да сам рад и мишљења других аутора које сам користио у овом раду назначио или цитирао и навео у списку литературе/референци који су саставни део овог рада;
- да сам добио све дозволе за коришћење ауторског дела који се у потпуности/целости уносе у предати рад и да сам то јасно навео;
- да сам свестан да је плагијат коришћење туђих радова у било ком облику (као цитата, парафраза, слика, табела, дијаграма, дизајна, планова, фотографија, филма, музике, формула, веб сајтова, компјутерских програма и сл.) без навођења аутора или представљање туђих ауторских дела као мојих, кажњиво по закону (Закон о ауторском и сродним правима, Службени гласник Републике Србије, бр. 104/2009, 99/2011, 119/2012), као и других закона и одговарајућих аката Универзитета у Новом Саду;
- да сам свестан да плагијат укључује и представљање, употребу и дистрибуирање рада предавача или других студената као сопствених;
- да сам свестан последица које код доказаног плагијата могу проузроковати на предати рад и мој статус;
- да је електронска верзија рада идентична штампаном примерку и пристајем на његово објављивање под условима прописаним актима Универзитета.

Нови Сад, **16.10.2026**. Потпис студента

_____

УНИВЕРЗИТЕТ У НОВОМ САДУ ● **ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА**
21000 НОВИ САД, Трг Доситеја Обрадовића 6

ИЗЈАВА О НЕПОСТОЈАЊУ СУКОБА ИНТЕРЕСА

Изјављујем да нисам у сукобу интереса у односу ментор-кандидат и да нисам члан породице (супружник или ванбрачни партнер, родитељ или усвојитељ, дете или усвојеник), повезано лице (крвни сродник ментора/кандидата у правој линији, односно у побочној линији закључно са другим степеном сродства, као ни физичко лице које се према другим основама и околностима може оправдано сматрати интересно повезаним са ментором или кандидатом), односно да нисам зависан/на од ментора/кандидата, да не постоје околности које би могле да утичу на моју непристрасност, нити да стичем било какве користи или погодности за себе или друго лице било позитивним или негативним исходом, као и да немам приватни интерес који утиче, може да утиче или изгледа као да утиче на однос ментор-кандидат.

У Новом Саду, дана _____

Ментор

_____

Кандидат

_____

Слике

Слика 1: Графички приказ примера мапе 14
Слика 2: Графички приказ решеног примера мапе 15
Слика 3: Блок дијаграм виртуелне платформе 19
Слика 4: Графички приказ коначне путање решене мапе 22
Слика 5: Графички приказ мапе која представља излаз акцелератора 23
Слика 6: Ток података у акцелератору 26
Слика 7: Блок дијаграм акцелератора са паралелизацијом 27
Слика 8: Блок дијаграм читавог система на Zybo плочи 30
Слика 9: Графички приказ критичне путање на RTL нивоу 31
Слика 10: Графички приказ предлога увођења додатног регистра 35

Табеле

Табела 1: Поља класе Nodes 13
Табела 2: Сентинел вредности distance поља 13
Табела 3: Резултати профајлирања Gprof алатом 16
Табела 4: Битска анализа помоћних промењивих 18
Табела 5: Регистарска мапа акцелератора у виртуелној платформи 20
Табела 6: Регистарска мапа акцелератора са AXI омотачем 29
Табела 7: Поређење времена по итерацији процењеног на системском и хардверском нивоу 32
Табела 8: Поређење процењеног и оствареног искоришћења ресурса 32
Табела 9: Процене времена по итерацији са повећаним фактором паралелизације 34
Табела 10: Преглед извршених провера 37

Скраћенице

ESL – *electronic system level,* Електронски системски ниво
RTL – *register transfer level,* Регистарски ниво преноса
IP – *intellectual property*, Интелектуална својина (готов хардверски модул)
GBFS – *greedy best-first search,* Похлепна (први – најбољи) претрага
TLM – *transaction level modeling,* Моделовање на нивоу трансакција
BRAM – *block random access memory,* Блок меморија са случајним приступом
HLS – *high level synthesis,* Синтеза високог нивоа
VHDL – *very high speed integrated circuit hardware description language,* Језик за опис хардвера
FPGA – *field-programmable gate array,* Програмабилно логичко коло
CPU – *central processing unit,* Централна процесорска јединица
ASMD – *algorithmic state machine with datapath*, Алгоритамска машина стања са путањом података
AXI – *advanced extensible interface*, Напредни проширив интерфејс
FSM – *finite state machine*, Коначни аутомат
UVM – *universal verification methodology*, Универзална методологија верификације
LUT – *look-up table*, Табела претраживања
FF – *flip-flop*, Флип-флоп / Бистабилно коло
DSP – *digital signal processing*, Дигитална обрада сигнала

Садржај

1. Увод ... 11

2. Софтверска спецификација и алгоритам претраге ... 12

2.1 Greedy Best-First Search (GBFS) алгоритам ... 12

2.2 Софтверска спецификација ... 13

2.2.1 Опис главних функција апликације ... 14

2.2.2 Графички приказ улаза и излаза алгоритма ... 14

3. Пројектовање на системском нивоу и виртуелна платформа ... 16

3.1 Анализа резултата профајлирања ... 16

3.2 Битска анализа ... 17

3.2.1 Битска анализа помоћних промењивих ... 17

3.2.2 Укупна потребна меморија ... 18

3.3 Виртуелна платформа ... 18

3.4 Ток трансакција у TLM моделу ... 19

3.5 Процена перформанси и трајање једне итерације ... 20

3.6 Закључна разматрања системског пројектовања ... 21

4. Хардверска имплементација акцелератора ... 22

4.1 Пример улаза и излаза акцелератора ... 22

4.2 Прелазак са софтверског описа на RTL опис ... 24

4.3 Стања система ... 24

4.4 Ток података у систему ... 26

4.5 Оптимизације извршене на хардверском нивоу ... 27

4.6 Тест бенч и провера основне функционалности акцелератора ... 28

4.7 Имплементација AXI4-Lite интерфејса и коначни блок дизајн система ... 29

4.7.1 Регистарска мапа језгра са омотачем и прекид ... 29

4.7.2 Коначни блок дизајн ... 30

5. Поређење системског и хардверског нивоа пројектовања ... 31

5.1 Анализа критичне путање ... 31

5.2 Поређење брзине рада ... 32

5.3 Поређење искоришћења ресурса ... 32

6. Предлози даљег рада ... 34

6.1 Предлог 1: имплементација проточне обраде ... 34

6.2 Предлог 2: повећање фактора паралелизације ... 34

6.3 Предлог 3: увођење кеширања ... 34

6.4 Предлог 4: скраћивање критичне путање ... 35

7. Преглед функционалне верификације и развоја драјвера ... 36

7.1 Функционална верификација ... 36

7.2 Развој драјвера и спуштање система на плочу ... 36

7.3 Преглед извршених провера ... 37

8. Закључак ... 38

9. Литература ... 39

1. Увод

Растућа сложеност дигиталних система померила је велики део инжењерског напора ка одлукама које се доносе много пре него што настане прва линија описа нивоа регистарских преноса (*RTL*, енгл. *register transfer level*). Начин на који се функционалност дели између процесора и наменског хардвера, организација меморије и процена временских захтева система, у великој мери одређују коначне перформансе и цену система. Одговор на овај изазов пружа пројектовање на системском нивоу (*ESL*, енгл. *electronic system level*), приступ у којем се хардвер и софтвер система најпре моделују на нивоу трансакција, кроз виртуелну платформу високог нивоа апстракције, чиме се архитектонске одлуке испитују и потврђују пре него што се уопште приступи имплементацији самог наменског хардвера. [1]

Практична реализација овог приступа заснива се на *SystemC* библиотеци програмског језика *C++,* и методологији моделовања на нивоу трансакција (*TLM, енгл. transaction level modeling*), којом се сигнали и временски токови апстрахују на ниво трансакција, уз довољно веродостојности да процена перформанси остане поуздана. [1][2] Софтверска спецификација се најпре профајлира, чиме се утврђује који њен део захтева највише времена извршавања, па управо то уско грло система, постаје природан кандидат за убрзавање у наменском *IP (енгл. intellectual property)* блоку, док остатак апликације наставља да се извршава на процесору.

Предмет овог рада јесте системско пројектовање и хардверска имплементација акцелератора за проналажење путање на дводимензионалној мапи димензија 100×100 чворова, заснованог на *GBFS* (енгл. *greedy best-first search*) алгоритму. У првој фази израђене су софтверска спецификација алгоритма и виртуелна платформа на системском нивоу, моделована у *SystemC*-у, док је у другој фази акцелератор реализован као хардверски опис у *VHDL* (*енгл. very high speed integrated circuit hardware description language*) језику и интегрисан у комплетан систем на *Zybo* плочи. Слични покушаји хардверског убрзања сродних алгоритама претраге путање, попут *A** алгоритма, показали су значајна убрзања у односу на софтверску имплементацију [4].

Рад прати ток пројектовања од алгоритма до система на плочи. Најпре су представљени *GBFS* алгоритам и софтверска спецификација, на основу које је профајлирањем извршено хардверско-софтверско партиционисање и израђена виртуелна платформа у *SystemC*-у. Затим је описана хардверска имплементација акцелератора у *VHDL*-у, његова интеграција са процесорским системом и поређење остварених перформанси са проценама са системског нивоа. На крају су дати предлози даљег рада, преглед функционалне верификације и развоја драјвера, током којих су уочене и исправљене две грешке у хардверу, и закључак.

2. Софтверска спецификација и алгоритам претраге

Основа за развој хардверског акцелератора је софтверска апликација написана у програмском језику *C++*. Њен основни задатак је проналажење путање на дводимензионалној мапи коришћењем *GBFS* алгоритма претраге.

2.1 *Greedy Best-First Search* (*GBFS*) алгоритам

Greedy Best-First Search представља хеуристички алгоритам претраге који у сваком кораку за експанзију бира онај чвор који, према хеуристичкој процени, има најмању преосталу удаљеност до циљног чвора. Формална спецификација алгоритма ослања се на функцију евалуације дефинисану као ***f(n) = h(n)***, где је ***n*** тренутни чвор, а ***h(n)*** хеуристичка функција која процењује трошак најкраћег пута од тог чвора до циља. [3]

Почев од стартног чвора, алгоритам израчунава хеуристике суседних чворова и затим из скупа свих до сада евалуираних чворова бира следећи чвор. Као следећи узима се чвор са најнижом ненултом хеуристиком. Уколико постоји више чворова исте хеуристичке вредности, узима се први на који се наиђе у претрази евалуираних чворова.

С обзиром на то да се алгоритам примењује на дводимензионалној мрежи (енгл. *grid*) димензија 100×100 чворова, кретање је дозвољено у осам праваца (хоризонтално, вертикално и дијагонално). Хеуристика може се тумачити као цена одређеног корака. Латерални корак мора бити јефтинији од дијагоналног, али два латерална морају бити скупља од једног дијагоналног. На тај начин дијагонално кретање је преферирано, уколико је могуће. Коришћена формула за хеуристику гласи:

$$h(n) = 2 \times |dx - dy| + 3 \times min(dx, dy) \qquad (1)$$

У изразу (1) фигуришу $dx = |endX - x|$, односно апсолутна разлика x-координате и крајњег чвора, као и $dy = |endY - y|$, односно апсолутна разлика y-координате посматраног и крајњег чвора.

За тежинске факторе, односно „цене“, одабране су вредности 2 за латералне кораке, и 3 за дијагоналне. Ове вредности задовољавају претходно наведене услове, а истовремено су и најмања целобројна комбинација која их задовољава. Та чињеница је од интереса јер значи да ће израчунате вредности хеуристика бити што је могуће мање, и самим тим захтевати мањи број бита за репрезентацију у хардверу. На мапи од 10 хиљада чворова, сваки уштеђени бит по чвору доноси глобалну уштеду од 10 kb.

У исто време, релевантна је чињеница да ће све хеуристике израчунате овом формулом бити целобројне вредности, што поједностављује хардверску имплементацију јер се избегава рад са бројевима са покретним или фиксним зарезом.

2.2 Софтверска спецификација

Софтверска апликација која је основа за овај пројекат је написана у *C++* програмском језику. Основна класа која енкапсулира чворове јесте *Nodes* – класа основних градивних јединица мапе. Мапа на којој се алгоритам извршава је дводимензионална мрежа (енгл. *grid*) димензије 100×100 чворова. Координате чвора се крећу у опсегу [0, 99] по обе осе. Алгоритам проналази путању од почетног до циљног чвора. Поља *Nodes* класе јесу:

Поље	Тип	Опис
x	int	X координата чвора у мрежи (0–99)
y	int	Y координата чвора у мрежи (0–99)
distance	int	Удаљеност чвора од циља
parentX	int	X координата родитељског чвора (0–99)
parentY	int	Y координата родитељског чвора (0–99)

Табела 1: Поља класе Nodes

Поље *distance* је заправо вредност хеуристике чвора. Промењиве *parentX* и *parentY*, односно координате родитељског чвора, су координате чвора чијом експанзијом је извршена евалуација хеуристике чвора.

На почетку, све хеуристике поља по којима је дозвољено кретање постављају се на вредност 0. Пошто удаљеност између два чвора не може бити мања од 2 (један латерални корак), 0 је у овом контексту сентинел вредност која кодира чињеницу да хеуристика чвора није израчуната, те се ови чворови прескачу када се тражи следећи чвор за експанзију.

Поред вредности нула, постоје још 4 сентинел вредности *distance* промењиве које кодирају посебне врсте чворова:

Вредност	Опис
298	Зид (препрека) – поље/чвор по којем није дозвољено кретање
299	Почетни чвор (*Start*)
300	Циљни/крајњи чвор (*End*)
301	Чвор који је део финалне путање

Табела 2: Сентинел вредности distance поља

Ове вредности проверавају се у одређеним корацима алгоритма, како се не би посетио недозвољени чвор (зид) и како би се препознао крајњи чвор и завршила претрага. Поред тога, ове вредности користе се у функцији која исцртава графички приказ мапе како би се лако препознао тип чвора и обојио одговарајућом бојом.

2.2.1 Опис главних функција апликације

Следеће функције обухватају главне функционалне целине апликације. Прве две, *evaluateSurroundingNodes( )* и *moveNode( )* имплементирају два корака који заједно чине једну итерацију претраге мапе, односно потребне операције за прелаз из једног чвора у други.

evaluateSurroundingNodes(int x, int y): Испитује осам суседних чворова тренутног чвора (x, y). За сваки непосећени чвор са нултом хеуристиком израчунава хеуристичку вредност и уписује је у поље distance.

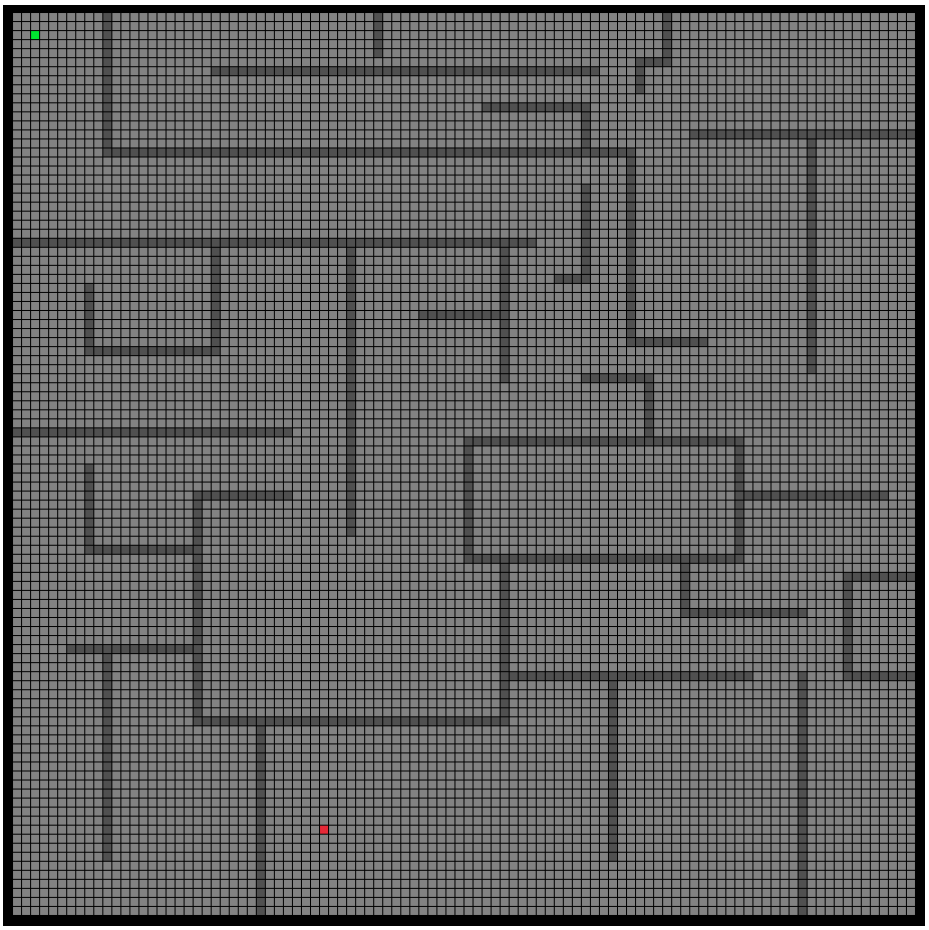
moveNode( ): Претражује целу мрежу и проналази непосећени чвор са најмањом позитивном вредношћу хеуристичке функције. Означава тренутни чвор као посећен и прелази на наредни чвор. Ова функција је уско грло система и главни кандидат за хардверско убрзавање.

finalPath( ): Реконструише пронађену путању тако што прати показиваче на родитељске чворове, почевши од циљног чвора, затим уназад све до почетног. Сваки чвор на путањи добија вредност 301 како би се могао препознати при исцртавању графичког приказа мапе.

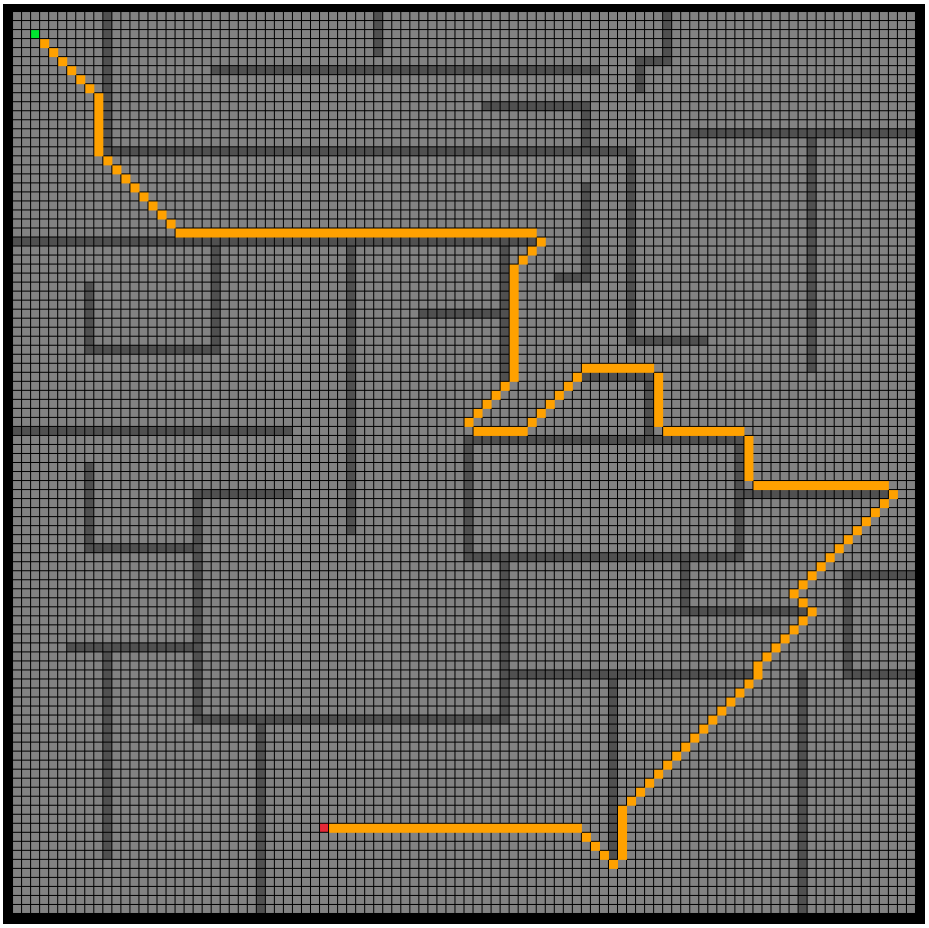
display(Nodes grid[][GridSize], int GridSize): Графички приказује тренутно стање мреже.

2.2.2 Графички приказ улаза и излаза алгоритма

Следеће две слике представљају пример графичког приказа једне мапе у почетном стању и мапе у решеном стању, на којој је означена коначна путања.



Слика 1: Графички приказ примера мапе



Слика 2: Графички приказ решеног примера мапе

Зидови, односно недозвољена поља, обојена су тамносивом бојом. Слободна поља, односно поља по којима је дозвољено кретање, обојена су светлосивом бојом. Почетно поље обојено је зеленом, а циљ црвеном бојом. На приказу решене мапе (*Слика 2*) може се приметити природа похлепног алгоритма, који у сваком кораку тежи да се приближи циљу, често по цени непотребног бочног кретања у одређеним сегментима, што доводи до одступања од најкраће могуће путање.

3. Пројектовање на системском нивоу и виртуелна платформа

Израда виртуелне платформе захтева одређивање функционалних блокова система. Како би се то постигло, најпре је неопходно дефинисати који део прорачуна и операција из апликације се реализује у хардверу, а који део остаје у софтверу, односно извршава се на процесору опште намене. Премештањем прорачуна у наменски *IP* блок и применом различитих метода оптимизације могуће је постићи значајно убрзање. Због тога, најсмисленије је у хардвер изместити део прорачуна који се неефикасно извршава на процесору. Одређивање тог дела прорачуна постиже се профајлирањем.

3.1 Анализа резултата профајлирања

Профајлирање урађено је уз помоћ *Gprof* алата. У наставку налази се табела са резултатима профајлирања софтверске апликације, у којој су приказани процентуални удели укупног времена извршавања за све функције као и број позива.

Функција	% времена	Број позива
moveNode()	75,00 %	1.742
Nodes::getDistance()	12,50 %	16.597.825
evaluateSurroundingNodes()	< 0,01 %	1.742
display()	12,50 %	2
finalPath()	< 0,01 %	1

Табела 3: Резултати профајлирања Gprof алатом

Првенствено, јасно је да *moveNode( )* функција представља очигледног кандидата за убрзавање издвајањем у наменски хардверски блок. Међутим, била би грешка ту се зауставити и узети само ову функцију у разматрање. Размотримо шта би била последица такве организације. При сваком преласку из једног чвора у други, процесор би прво урадио експанзију чвора позивом *evaluateSurroundingNodes( )* функције. Затим би те резултате морао некако саопштити *IP* блоку, и уписати их у меморију како би се мапа освежила. Тек тада би акцелератор (*IP)* могао да започне претрагу мапе за следећим чвором, пређе у следећи чвор, и поново чека процесор да изврши експанзију тог чвора. Ова уска међузависност ове две функције огледа се у идентичном броју позива у табели, тачно 1742 позива.

Метода *Nodes::getDistance( )* представља приступање *distance* пољу, па иако има висок број позива, из угла оптимизације нема потенцијала.

Због уске повезаности функција *moveNode()* и *evaluateSurroundingNodes()* одлучено је да се заједно издвоје у акцелератор, односно *IP* блок. Тиме се комуникација између процесора и акцелератора своди на 2 трансакције. На самом почетку, процесор ће обавити потребне иницијализације и уписати мапу у меморију. Акцелератор приступа меморији и итеративно обавља прорачуне двеју функција, док не дође до краја мапе. На крају процесор из меморије ишчитава коначну путању полазећи од крајњег чвора. Из алгоритамске перспективе, сав захтеван прорачун померен је у акцелератор, док процесор обавља прорачуне за које се може сматрати да се извршавају у константном времену.

3.2 Битска анализа

У оквиру хардверске имплементације неопходно је дефинисати прецизан број бита потребних за репрезентацију свих релевантних података. Тако се минимизује количина потребних ресурса. Ова анализа такође даје одговор на питање да ли планирани систем уопште може да се реализује користећи *Zybo* плочу. За потребе овог пројекта, урађена је статичка битска анализа на основу унапред познатих опсега вредности промењивих.

Сваки чвор мапе представљен је објектом класе *Nodes*. Информације о координатама X и Y (у опсегу 0-99) се не чувају експлицитно, већ се њихова вредност користи за меморијске адресе према формули датој у изразу (2).

$$Adresa = X \times 100 + Y \tag{2}$$

Подаци о сваком чвору кодовани су у 24-битну структуру која се смешта у меморију, са следећим распоредом:

- **distance (9 бита)**: Чува израчунату хеуристичку вредност (1–297) или специјалне кодоване вредности. Видети поглавље 2.2 за могуће сентинел (кодоване) вредности.
- **parentX [14:8] (7 бита)**: X координата родитељског чвора (опсег 0–99).
- **parentY [7:1] (7 бита)**: Y координата родитељског чвора (опсег 0–99).
- **visited [0] (1 бит)**: Индикатор (булова вредност) који говори да ли је чвор већ посећен.

Оваква битска оптимизација резултује укупним меморијским захтевом од око 30 kB за целу мапу од 10.000 чворова, што је погодно за смештање у меморију на *FPGA (енгл. field-programmable gate array)* чипу који садржи 270 kB *BRAM*-а (енгл. *block random access memory*).

Индикаторски бит *visited*, у софтверској апликацији чувао се у засебном низу који је служио за бележење посећености чворова користећи булове типове. За хардверску имплементацију он је једноставно додат као још један бит у структури која описује чвор. Сврха овог бита је једноставна: при претрази за следећим чвором, проверава се *visited* бит, и одбијају се чворови код којих је 1. На тај начин спречава се било који тип мртвих петљи, и обезбеђен је механизам завршавања алгоритма уколико је мапа нерешива.

3.2.1 Битска анализа помоћних промењивих

Поред бита потребних за кодовање чворова, односно мапе, потребно је кодовати и одређене помоћне промењиве које се користе у алгоритму. Ова анализа је од мањег значаја јер је број бита потребан за помоћне промењиве два реда величине мањи од количине бита везаних за чворове. Поред тога, ове промењиве су подложне променама током развоја хардверског дизајна, па је скоро немогуће унапред одредити тачан број потребних бита. Процена је извршена статичком анализом софтверске спецификације, и представљена у следећој табели.

Промењива	Функција	Број бита	Опсег	Образложење
distance	evaluateSurroundingNodes()	9 бита	0 – 297	Највећа могућа удаљеност је између два наспрамна угла мапе, на пример (0,0) и (99,99) и износи 99 × 3 = 297
minDistance	moveNode()	9 бита	0 – 297	
dist	moveNode()	9 бита	0 – 297	
nextX	moveNode()	7 бита	0 – 99	100 вредности
nextY	moveNode()	7 бита	0 – 99	100 вредности
currentX	moveNode()	7 бита	0 – 99	100 вредности
currentY	moveNode()	7 бита	0 – 99	100 вредности
Укупно	**55 бита**			

Табела 4: Битска анализа помоћних промењивих

3.2.2 Укупна потребна меморија

На основу спроведене анализе, укупна меморија потребна за чување комплетног стања мреже износи:

$$24 \text{ бита} \times 10.000 \text{ чворова} + 55 \text{ бита} = 240.055 \text{ бита} \approx 30.007\ B < 31\ kB \quad (3)$$

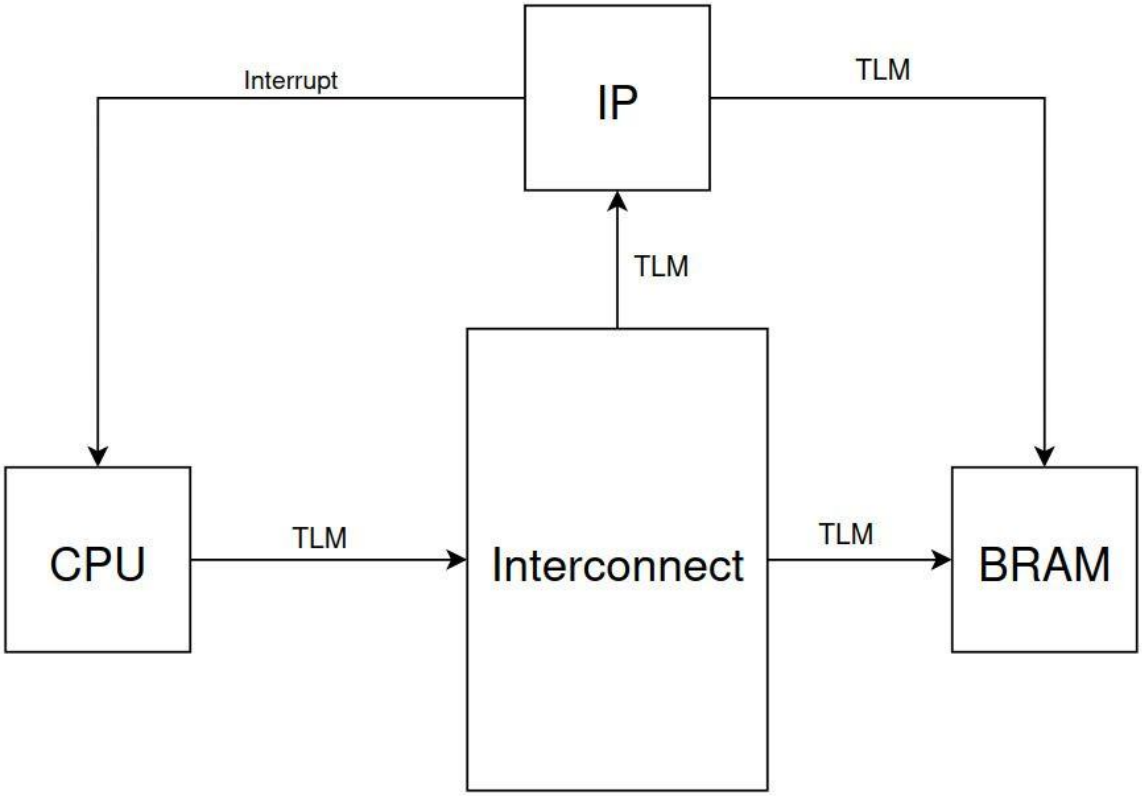
Овај податак је релевантан за димензионисање BRAM меморије која ће бити коришћена у хардверском акцелератору. Укупна потребна меморија, према изразу (3), од испод 31 kB је унутар прихватљивог опсега за реализацију на *Zybo* плочи.

3.3 Виртуелна платформа

Виртуелна платформа представља апстрактни модел система реализован у програмском језику *C++*, уз коришћење *SystemC* библиотеке. Да би се омогућила брза симулација примењена је *TLM* методологија, специфично *Loosely-timed* приступ. [2] Сви физички сигнали на магистралама су апстраховани на ниво једноставних преноса података уз дефинисано временско кашњење.

Архитектура виртуелне платформе састоји се од четири главна модула:

1. **CPU (Процесор):** Модул који генерише мапу, поставља препреке, иницијализује BRAM, конфигурише IP блок и, по пријему прекида, реконструише финалну путању.
2. **IP (Хардверски акцелератор):** Модул који у себи садржи логику која обавља операције *moveNode( )* и *evaluateSurroundingNodes( )* функција. Када добије стартни сигнал, покреће процес претраге у петљи.
3. **BRAM (Блок меморија):** Меморијски модул који чува стање свих 10.000 чворова. Има два *TLM* приступа како би му истовремено могли приступати и CPU (енгл. *central processing unit*) и IP блок. На исти начин, у хардверу постоје 2 *BRAM* приступа.
4. **Interconnect (Магистрала):** Модул који рутира *TLM* трансакције од процесора ка BRAM-у или IP блоку, на основу адресног декодирања.



Слика 3: Блок дијаграм виртуелне платформе

На слици 3 приказан је блок дијаграм виртуелне платформе. Стрелицама између блокова означене су трансакције. Смер стрелице указује на то који од два блока може да покрене трансакцију. Процесор покреће трансакције према *BRAM*-у и *IP*-у. *IP* покреће трансакције према *BRAM*-у, и може да шаље прекидни сигнал процесору.

3.4 Ток трансакција у *TLM* моделу

На почетку симулације процесор генерише мапу као дводимензионални низ чворова, поставља почетни чвор, циљни чвор и препреке, а затим је уписује у *BRAM*. За сваки од 10.000 чворова покреће се по једна трансакција типа *TLM_WRITE_COMMAND* преко *Interconnect* модула. Свака трансакција преноси једну 24-битну реч која описује чвор (поглавље 3.2).

Interconnect модул на основу адресе прослеђује трансакцију одговарајућем модулу. Опсег 0x0000–0x270F припада *BRAM*-у, а опсег 0x2710–0x2712 регистрима *IP* блока. Трансакција ван ових опсега завршава се грешком адресирања.

Након иницијализације меморије процесор конфигурише *IP* блок преко његове регистарске мапе:

Регистар	Локална адреса	Глобална адреса	Податак
START_COORD_ADDRESS	0x0	0x2710	[13:7] startX, [6:0] startY
END_COORD_ADDRESS	0x1	0x2711	[13:7] endX, [6:0] endY
START_SIGNAL_ADDRESS	0x2	0x2712	упис покреће решавање

Табела 5: Регистарска мапа акцелератора у виртуелној платформи

Глобална адреса 0x2710 је број 10.000. Меморији је додељен адресни опсег 0–9999 за чување 10.000 чворова, а регистрима акцелератора три наредне слободне адресе. Сва три регистра су само за упис: *IP* блок прихвата искључиво трансакције уписа.

Подаци о позицији почетног и циљног чвора потребни су акцелератору да би започео решавање мапе. Када их процесор не би уписао на овај начин, акцелератор би пре почетка решавања морао да претражи целу мапу како би пронашао та два чвора. Након уписа координата, процесор уписује произвољну вредност у регистар *START_SIGNAL_ADDRESS*. Акцелератор тај упис тумачи као стартни сигнал и покреће решавање мапе.

Током решавања *IP* блок итеративно извршава операције *evaluateSurroundingNodes( )* и *moveNode( )*. Меморији приступа директно, преко другог *TLM* приступа *BRAM*-а, без посредства *Interconnect* модула. Када пронађе циљни чвор, *IP* блок генерише краткотрајни импулс на прекидном сигналу *irq*. Процесор чека растућу ивицу тог сигнала, затим ишчитава свих 10.000 чворова из меморије, реконструише коначну путању и исцртава графички приказ решене мапе.

Прекидни сигнал *irq* није приказан у табели 5 јер није регистар доступан преко магистрале, већ сигнал директно повезан на процесор. У хардверској имплементацији регистарска мапа је проширена статусним регистром, а прекид је реализован као „лепљив“ (поглавље 4.7.1).

3.5 Процена перформанси и трајање једне итерације

Процена хардверских перформанси алгоритма извршена је помоћу алата *Vitis HLS* (*енгл. high level synthesis*). [6] Резултати синтезе, уз примењене оптимизације проточног извршавања (*pipeline*) и коришћење једног приступа *BRAM* меморије (јер акцелератор користи један приступ, док други користи процесор) показују да је могуће достићи минималну периоду такта од око 7,078 ns, док латенција по итерацији износи 10.007 циклуса такта.

Латенција по итерацији користи се као објективна мера пропусне моћи система, јер не зависи од мапе, већ само од имплементације и величине мапе. Ова мера заправо представља број тактова потребан да се изврши прорачун еквивалентан једном позиву двеју имплементираних функција.

На основу кашњења од 10.007 циклуса и периоде такта од 7,078 ns, време извршавања једне итерације износи приближно 70,83 µs. Користећи ове податке, може се одредити пропусна моћ (енгл. *throughput*) акцелератора од око 14.118 итерација у секунди. С обзиром на то да се у свакој итерацији претражује свих 10.000 чворова, ефективна пропусна моћ меморијског претраживања достиже више од 140 милиона прегледаних чворова у секунди.

3.6 Закључна разматрања системског пројектовања

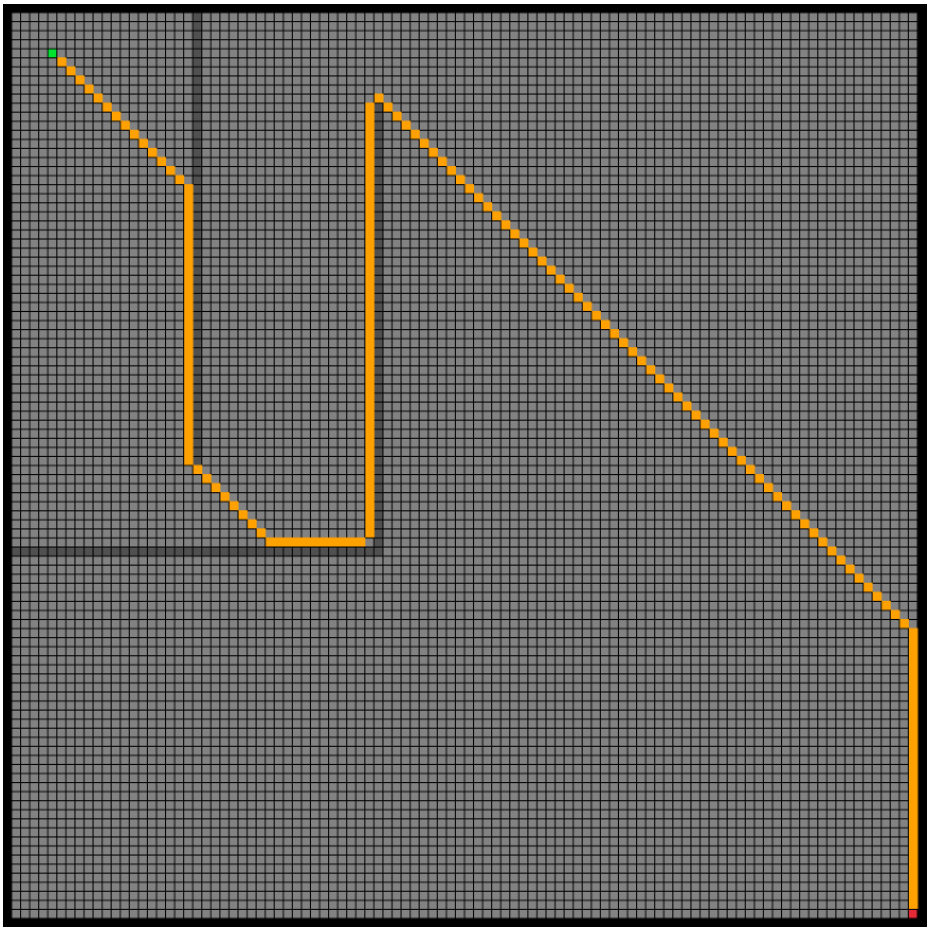
Развој виртуелне платформе у *SystemC*-у омогућио је рану верификацију организације и тока података система. Успешно је дефинисан распоред хардверско-софтверског партиционисања. Интеграцијом критичних функција у *IP* блок елиминисана су уска грла и растерећен је процесор. Поред функционалне потврде, платформа је дала циљ за перформансе система и обезбедила поуздан референтни модел као сигурну основу за каснију RTL имплементацију.

4. Хардверска имплементација акцелератора

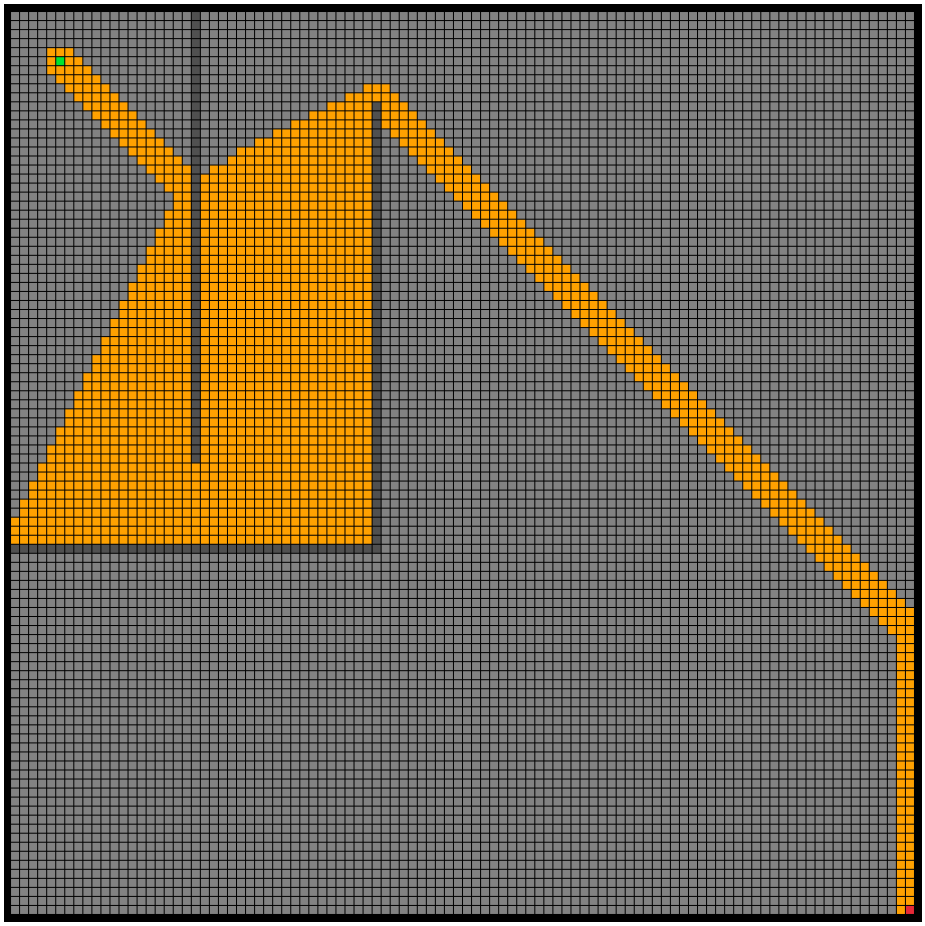
Након што је дефинисана виртуелна платформа и спроведена провера рада система на високом нивоу апстракције, следи процес превођења концептуалног модела у конкретну хардверску архитектуру. Главни циљ пројектовања наменског хардверског акцелератора јесте растерећење процесора, као и директно израчунавање алгоритма у хардверу на што ефикаснији начин. У оквиру овог поглавља обухваћено је пројектовање акцелератора на хардверском нивоу, односно прелаз са *ESL* на *RTL* ниво описа система.

4.1 Пример улаза и излаза акцелератора

Пре разматрања процеса пројектовања, корисно је стећи увид у то шта заправо представља излазни резултат акцелератора. Пошто екстракцију коначне путање обавља процесор, излаз акцелератора није путања. Следе два графичка приказа, који су креирани тек по завршетку пројектовања, али су корисни за разумевање стања мапе непосредно након што акцелератор заврши решавање.



Слика 4: Графички приказ коначне путање решене мапе



Слика 5: Графички приказ мапе која представља излаз акцелератора

Оба приказа креирана су над истом улазном мапом. Слика 4 представља већ познати излаз система у виду путање од старта до циља. На слици 5 путања још није одређена, јер тај посао обавља процесор, а не акцелератор. Поља која су обојена наранџастом бојом представљају претражене чворове. Претраженим чворовима називају се слободни (проходни) чворови, чија је хеуристичка вредност израчуната у току решавања мапе. Овај скуп чворова је унија чворова над којима је извршена експанзија и чворова који су им суседни (међусобне удаљености 2 или 3).

Разумевање ове разлике између излаза читавог система и излаза акцелератора потребно је како би се могло визуелно и интуитивно потврдити да акцелератор ради исправно. Уколико је наранџасти скуп чворова стигао до чвора суседног крајњем чвору јасно је да је циљ пронађен.

Графички приказ на слици 5 генерисан је апликацијом написаном у *C++* програмском језику која на основу текстуалног фајла који садржи стање *BRAM* меморије екстрахованог током симулације у *VHDL* тестном окружењу исцртава мапу.

4.2 Прелазак са софтверског описа на *RTL* опис

Како би се софтверски опис функција *moveNode( )* и *evaluateSurroundingNodes( )* у *C++* језику превео у опис нивоа регистарских трансфера у *VHDL*-у, спроведени су следећи кораци:

Прво су *C++* објектно-оријентисани конструкти преведени у *C* програмски језик. Низ објеката класе *Nodes* замењен је са више низова за индивидуална поља, а позиви метода су замењени њиховим телима која раде над тим низовима. Ово је урађено у циљу приближавања организације података облику који је природнији за хардвер.

Затим је извршено уклањање петљи из кода. *For* и *while* петље замењене су *if – goto* обрасцима. Као последњи корак израде пре креирања *RTL* описа нацртан је *ASMD* (енгл. *algorithmic state machine with datapath*) дијаграм [5].

Резултујући дигитални систем је машина стања са путањом података. Систем је затим описан у *VHDL*-у.

4.3 Стања система

Рад акцелератора управљан је коначним аутоматом са 13 стања. Првих седам стања обавља експанзију тренутног чвора (еквивалент функције *evaluateSurroundingNodes( )*), наредних пет претрагу мапе и прелазак у следећи чвор (еквивалент функције *moveNode( )*), а последње стање означава завршетак рада.

Поред прекидног импулса *irq*, језгро има два статусна излаза. Сигнал *busy* активан је док аутомат није у стању **IDLE**, односно док траје решавање мапе. Сигнал *solved_out* преноси вредност *solved* бита, па процесор по завршетку рада може да разликује решену од нерешиве мапе.

IDLE: Иницијално стање. Ресетују се *solved* бит и регистри *min_dist*, *next_x* и *next_y*. Чека се упис на контролну адресу (*cpu_addr* = "10" и *cpu_we* = '1'). Тада се *current_x* и *current_y* постављају на стартне координате и прелази се у **EVAL_START**.

EVAL_START: Помоћни регистри *eval_i* и *eval_j* постављају се на координате првог суседа (*current_x* − 1, *current_y* − 1), чиме почиње обилазак осам суседних чворова.

EVAL_READ: Проверава се да ли су индекси ван граница мапе (опсег 0–99) или одговарају тренутном чвору. Ако јесу, прелази се у **EVAL_NEXT**. У супротном се адреса суседа прослеђује *BRAM*-у и прелази се у **EVAL_WAIT**.

EVAL_WAIT: Такт кашњења неопходан услед синхроног читања из *BRAM* меморије.

EVAL_PROCESS: Анализира се прочитани чвор. Ако је то циљни чвор (*distance* = 300), у њега се уписују координате тренутног чвора као родитеља, поставља се *solved* бит и прелази се у **EVAL_WRITE**. Ако хеуристика чвора није рачуната (*distance* = 0), рачуна се хеуристика, уписује се заједно са координатама родитеља и прелази се у **EVAL_WRITE**. У осталим случајевима (**WALL**, **START** или већ евалуиран чвор) упис се прескаче и прелази се у **EVAL_NEXT**. Бит *visited* се у овом стању не мења.

EVAL_WRITE: Такт потребан за упис новог стања чвора у *BRAM* меморију.

EVAL_NEXT: Ако је *solved* бит постављен, прелази се у **DONE**. У супротном се индекси померају на следећег суседа и аутомат се враћа у **EVAL_READ**. Када су обрађени сви суседи, припрема се паралелна претрага мапе: четири показивача *pscan* постављају се на почетке четири сегмента меморије ($x = 0, 25, 50$ и 75), локални минимуми *min_dist1..4* се ресетују, сигнал *bram_parallel* поставља се на '1' и прелази се у **MOVE_READ_P**.

MOVE_READ_P: Координате четири показивача претварају се у 12-битне локалне адресе и истовремено прослеђују у четири *BRAM* банке.

MOVE_WAIT_P: Такт кашњења неопходан услед синхроног читања из *BRAM* меморије.

MOVE_PROCESS_P: За сваку од четири банке анализира се прочитани чвор. Ако се његове координате поклапају са тренутним чвором, садржај се чува у регистру *current_node*. Ако је чвор непосећен ($visited = 0$), валидан ($0 < distance < 298$) и има мању дистанцу од локалног минимума своје банке, ажурирају се *min_dist*, *next_x* и *next_y* те банке.

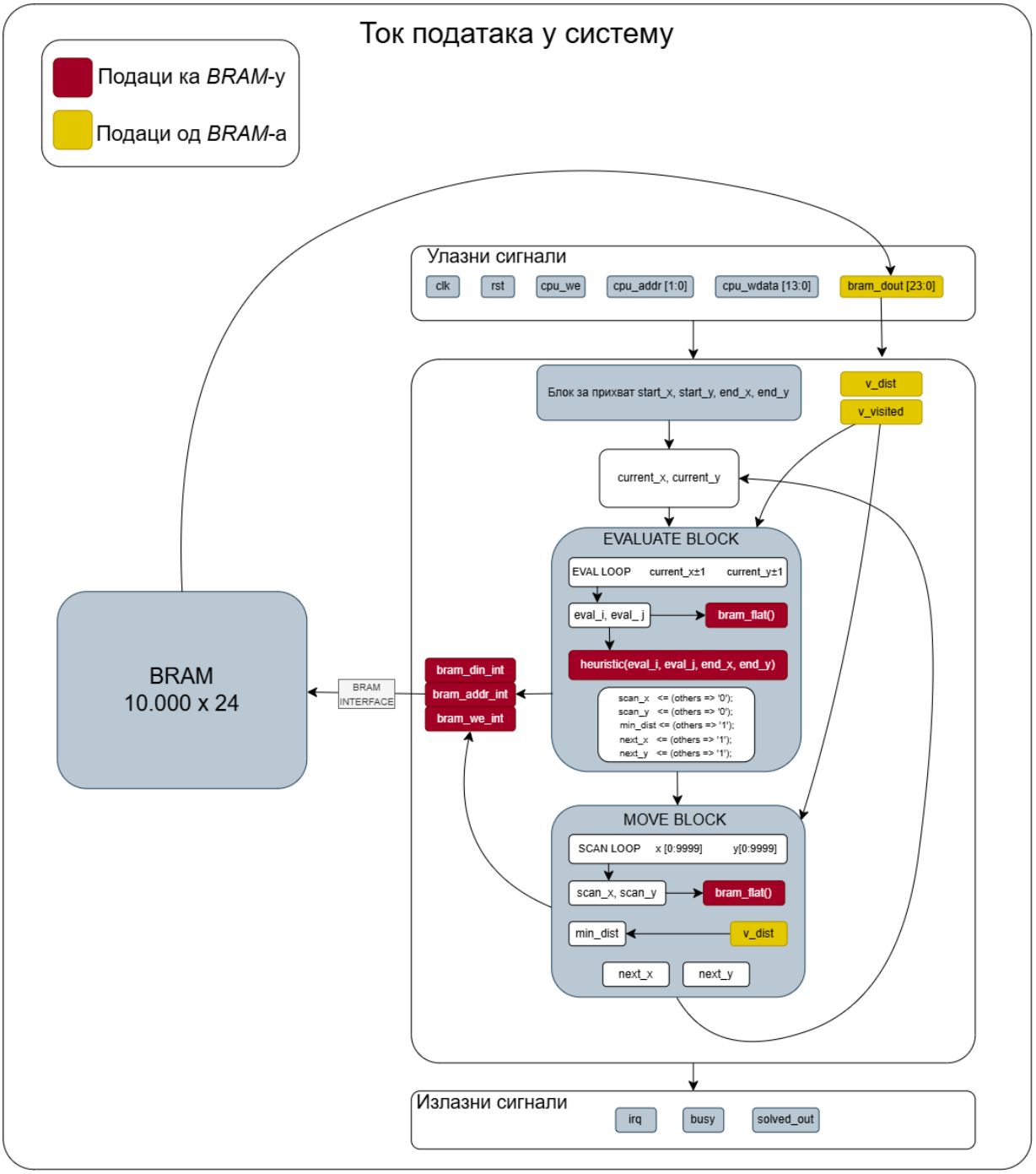
MOVE_NEXT_P: Сва четири показивача померају се на следећи чвор свог сегмента и аутомат се враћа у **MOVE_READ_P**. Када је обиђено свих 2500 чворова сегмента, од четири локална минимума бира се најмањи и уписује у *min_dist*, *next_x* и *next_y*. Сигнал *bram_parallel* враћа се на '0' и прелази се у **MOVE_NEXT**.

MOVE_NEXT: Ако *next_x* и *next_y* нису измењени (остали су на вредности 127), нема више валидних чворова, мапа је нерешива и прелази се у **DONE**. У супротном се садржај регистра *current_node* са постављеним *visited* битом уписује на адресу тренутног чвора, *current_x* и *current_y* преузимају вредности *next_x* и *next_y* и аутомат се враћа у **EVAL_START**.

DONE: Генерише једнотактни импулс на излазу *irq* језгра и враћа аутомат у **IDLE**. Импулс се у *AXI4-Lite* (енгл. *advanced extensible interface*) омотачу памти и одржава као „лепљиви“ прекид (поглавље 4.7.1).

4.4 Ток података у систему

Ток података кроз систем приказан је на следећем дијаграму:



Слика 6: Ток података у акцелератору

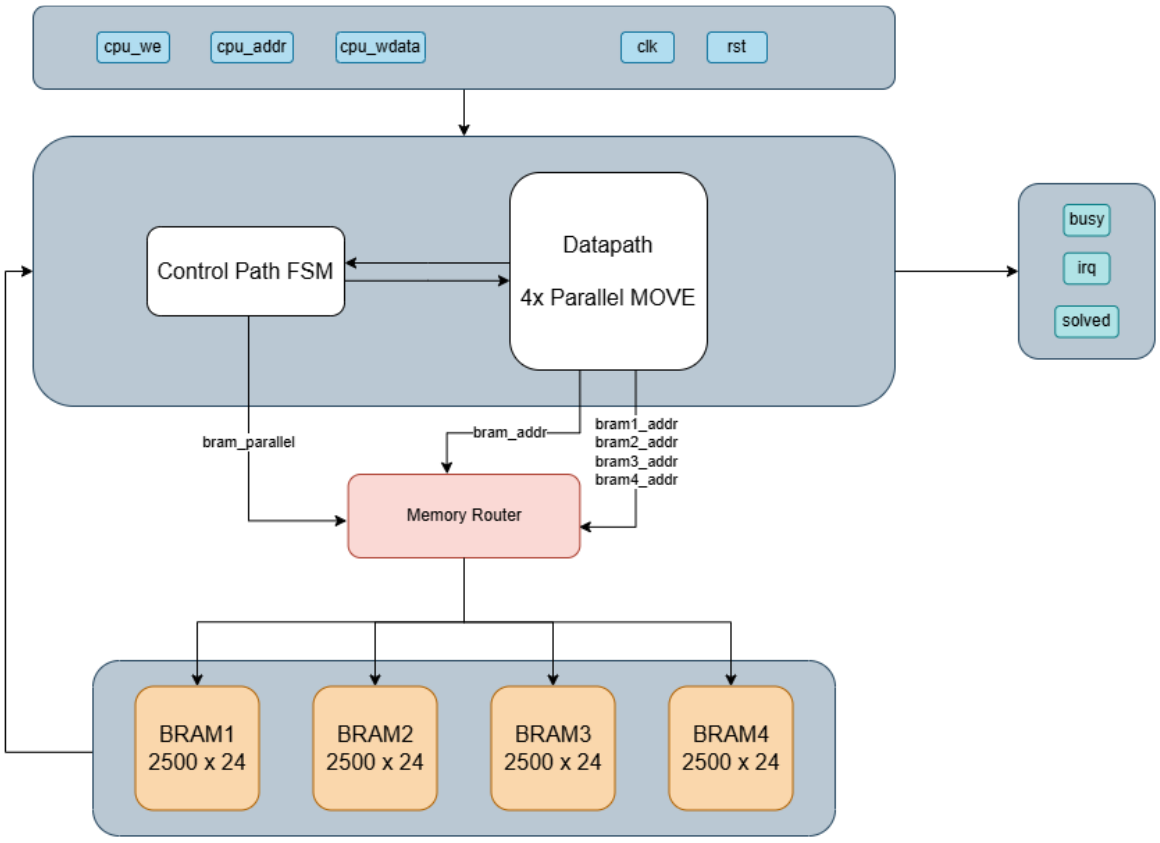
На дијаграму издвојене су 4 целине: улазни и излазни сигнали, *BRAM* меморија и централни блок који представља интерне прорачуне акцелератора. Акцелератор је унутар себе подељен на два функционална блока: *MOVE* и *EVALUATE* блок, који обављају прорачуне еквивалентне *moveNode( )* и *evaluateSurroundingNodes( )* функцијама.

Итеративност система графички се може препознати по току (стрелици) која представља пренос одређених координата следећег чвора у регистре координата тренутног чвора. Одатле *EVALUATE* блок започиње експанзију следећег чвора и алгоритам се итеративно наставља све до проналаска циља или до оног тренутка када понестане валидних чворова за експанзију. Дијаграм приказује основни ток података са једним током претраге. Паралелизација претраге приказана је на слици 7.

4.5 Оптимизације извршене на хардверском нивоу

Директна имплементација прорачуна у наменском хардверу пружа могућност убрзања рада система. Поред тога извршене су мање оптимизације попут паковања података у 24-битне пакете организоване по чворовима како би се смањио број приступа меморији. Додат је *current_node* регистар, који складишти тренутни чвор већ у тренутку претраге мапе, чиме се избегава додатно читање из меморије пре постављања *visited* бита на јединицу. На тај начин уштеђен је један такт по итерацији, што у зависности од мапе доноси временску уштеду од неколико стотина до више хиљада циклуса такта.

Најзначајнија оптимизација извршена у хардверу јесте четворострука паралелизација претраге мапе, односно *MOVE* блока. Ова оптимизација се једноставно уводи као последица организације меморије. Пошто је свака реч у меморији један чвор, меморија се може поделити у 4 сегмента који чувају по 2500 чворова. Сегменти се могу индивидуално претраживати, и затим по цени једног додатног такта, од четири чвора са локално минималним хеуристикама бира се онај са најнижом хеуристиком једноставним поређењем.



Слика 7: Блок дијаграм акцелератора са паралелизацијом

Ова оптимизација са собом повлачи нешто већи утрошак ресурса на чипу, јер се хардвер за претрагу мапе реплицира четири пута, али доноси скоро четвороструко убрзање претраге.

На дијаграму приказан је и блок под називом *Memory Router*. Овај блок морао је бити уведен како би *EVALUATE* блок и даље могао да приступа меморији преко глобалних адреса (0-9999). За ту потребу је уведен и додатни контролни сигнал *bram_parallel* који одређује да ли се меморији прослеђује декодирана глобална 14-битна адреса или се паралелно приступа свим блоковима меморије користећи 12-битне локалне адресе.

4.6 Тест бенч и провера основне функционалности акцелератора

Верификација основне функционалности акцелератора изведена је помоћу *VHDL* тест бенча. У њему се инстанцирају тестирани IP блок и четири BRAM меморије које заједно чине меморију мапе. Пошто у симулацији не постоји процесор, тест бенч га опонаша, тако што уписује потребне вредности у регистре и покреће IP, чека прекид и снима садржај меморије у текстуални фајл, како би се користио за касније анализе.

У наставку налази се ток симулације:

Иницијализација – сигнал *rst_ip* постављен на '0' (ресет активан), и сви уписни сигнали (*wea1..wea4*) су неактивни.

Учитавање мапе – тест бенч отвара фајл дефинисан константом *BRAM_INIT* и чита из њега линију по линију помоћу *VHDL std.textio* библиотеке. Свака линија се конвертује у 24-битни *std_logic_vector* и уписује на одговарајућу адресу одговарајуће банке (опсег адреса од 0 до 9999 се пресликава на банке 1–4, по 2500 адреса у сваку банку).

Активација *IP*-а – након учитавања, *rst_ip* прелази у '1', чиме *IP* блок излази из ресета и *FSM* (енгл. *finite state machine*) постаје активан (стање *IDLE*).

Упис *START* регистра – преко *cpu_addr* = "00" уписују се координате почетног чвора.

Упис *END* регистра – преко *cpu_addr* = "01" уписују се координате циљног чвора.

Старт порука – са *cpu_addr* = "10" и *cpu_we* = '1' преводи *FSM* из *IDLE* у *EVAL_START* и покреће решавање мапе.

Чекање на прекид – тест бенч чека *irq_ip* = '1', што одговара тренутку када *FSM* уђе у стање *DONE* (мапа решена или утврђено да је нерешива).

Снимање стања меморије – по пријему прекида, тест бенч редом чита свих 10.000 адреса и уписује прочитане 24-битне речи, линију по линију, у фајл дефинисан константом *BRAM_DUMP*.

Тестиране су три мапе, и у свим инстанцама акцелератор је радио исправно и према очекивањима. Исправност је евалуирана на основу визуелног приказа стања меморије по завршетку симулације. Пример оваквог приказа налази се у поглављу 4.1 на слици 5. Верификација граничних случајева попут нерешивих мапа остављена је за крајње фазе пројекта, у склопу функционалне верификације. Тест бенч инстанцира само језгро акцелератора, па се у њему прекид појављује као једнотактни импулс. Статусни регистар и „лепљиви“ прекид део су *AXI4-Lite* омотача (поглавље 4.7.1) и проверени су на плочи, кроз рад драјвера (поглавље 7.2).

4.7 Имплементација *AXI4-Lite* интерфејса и коначни блок дизајн система

Акцелераторско *IP* језгро заједно са *AXI4-Lite slave* омотачем запаковано је како би се могло инстанцирати у блок дизајну. [7]

4.7.1 Регистарска мапа језгра са омотачем и прекид

Регистар	***AXI* адреса**	**Податак**
START	0x40010000	startX, startY
END	0x40010004	endX, endY
CONTROL	0x40010008	било који упис тумачи се као стартни сигнал
STATUS	0x4001000C	Три бита: [0] *done* (лепљив), [1] *busy*, [2] *solved*

Табела 6: Регистарска мапа акцелератора са AXI омотачем

Подаци о позицији почетног чвора (регистар *START*) и крајњег, односно циљног чвора (регистар *END*) потребни су акцелератору да би започео решавање мапе. У оба регистра координата *x* смешта се у битове [13:7], а координата *y* у битове [6:0]. Када процесор не би уписао ове податке, акцелератор би пре почетка решавања морао да претражи целу мапу како би пронашао стартни и циљни чвор.

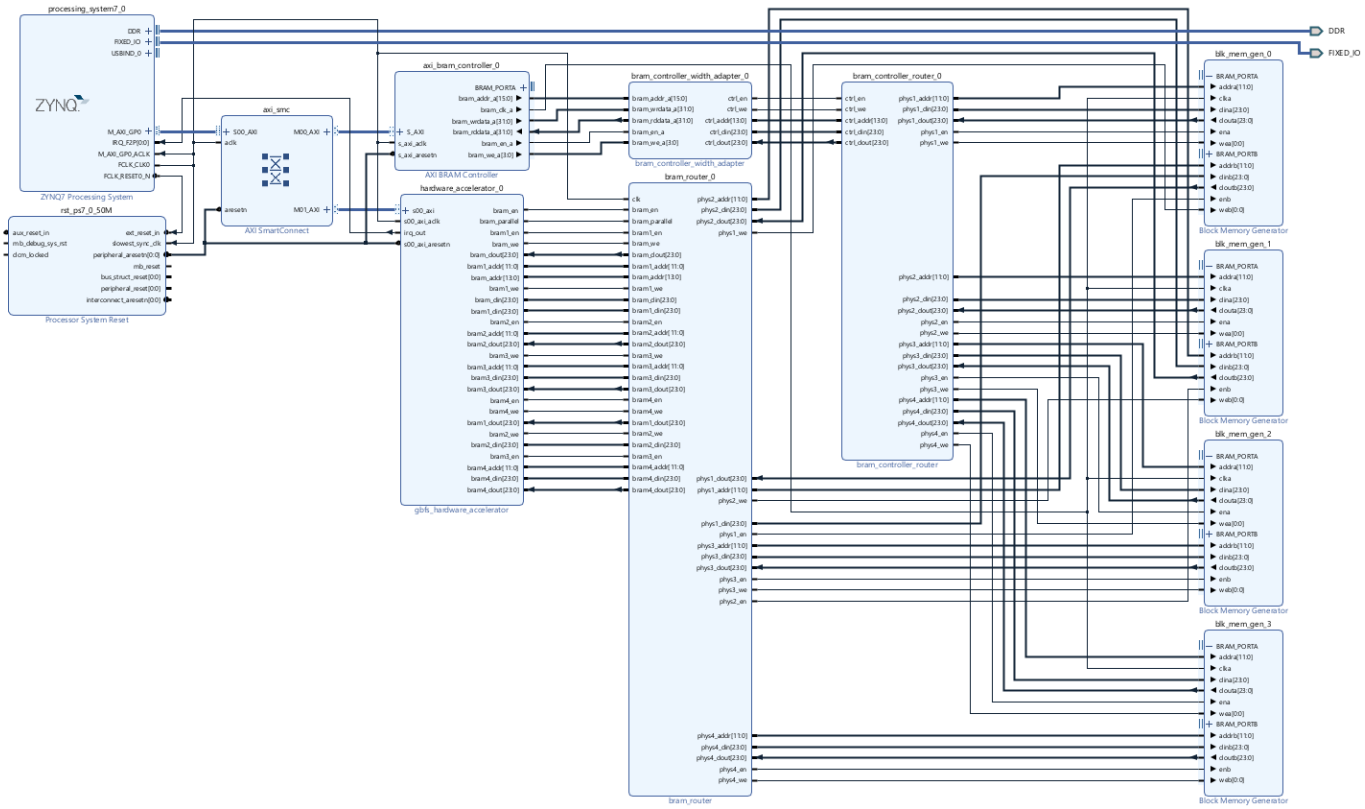
Након уписа координата, процесор уписује произвољну вредност у регистар *CONTROL*. Акцелератор тај упис тумачи као стартни сигнал и започиње решавање мапе. По завршетку решавања, акцелератор поставља бит *done* статусног регистра (*STATUS*) и активира прекидни сигнал *irq*. Прекид је „лепљив“: сигнал *irq* остаје на јединици све док процесор не обрише бит *done* уписом јединице у тај бит. Бит *solved* статусног регистра говори да ли је циљ пронађен или је мапа нерешива, јер се прекид генерише у оба случаја. Бит *busy* показује да ли акцелератор тренутно ради. По пријему прекида процесор брише *done* бит, проверава бит *solved*, а затим ишчитава коначну путању из меморије и црта графички приказ решене мапе.

Када се ова регистарска мапа (Табела 6) упореди са регистарском мапом која је предвиђена на системском нивоу (Табела 5, поглавље 3.4), прва три регистра имају исте улоге. Статусни регистар прави разлику, јер је он накнадно додат у хардверу, пре свега како би се имплементирао „лепљиви прекид“.

Прекидни сигнал *irq* није приказан у табели 6 јер није регистар доступан преко *AXI* магистрале, већ је директно повезан на прекидни улаз процесора.

4.7.2 Коначни блок дизајн

У коначном блок дизајну, додатно су имплементирана помоћна *IP* језгра за рутирање *BRAM* интерфејса од процесора и акцелератора ка меморији. Затим је инстанциран адаптер ширине, који линије података (32 бита) и адреса (16 бита) са стране *AXI-BRAM* контролера, прилагођава меморији, која очекује 24 бита за податке и 14 бита за адресе. Од 32-битне адресе прослеђују се бити [15:2] јер последња два бита служе за адресирање бајта, а меморији се приступа искључиво на нивоу речи. У наставку налази се приказ блок дизајна система на *Zybo FPGA* чипу, заједно са свим језгрима, *Zynq-7000* процесорским системом, *AXI* магистралом и инстанцираним блоковима меморије.



Слика 8: Блок дијаграм читавог система на Zybo плочи

5. Поређење системског и хардверског нивоа пројектовања

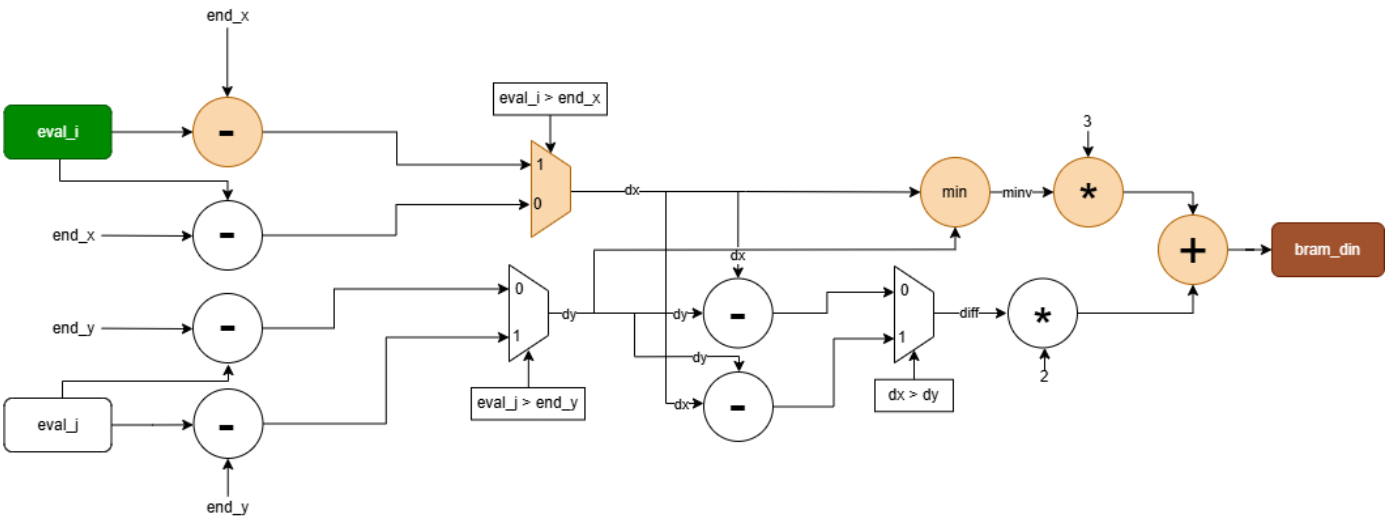
Анализом критичних путања у *Vivado* алату, процењена је минимална периода такта имплементираног акцелератора од 9,9 ns. Према томе, максимална радна учестаност система износи приближно 101 MHz.

Користећи *HLS* алат, процењена је минимална периода такта у вредности 7,078 ns, односно учестаност од 141 MHz. Дакле, процењена учестаност након имплементације износи око 71,5 % од максималне учестаности процењене *HLS* алатом. Максималну учестаност реализованог акцелератора ограничава критична путања у *EVALUATE* блоку, која је анализирана у следећем одељку. Различит приступ имплементацији *MOVE* блока утиче на број тактова по итерацији, што је разматрано у одељку 5.2.

5.1 Анализа критичне путање

У прегледу временске анализе имплементације, *Vivado* алат као критичну истакао је путању између *eval_i_reg* регистра и *bram_din* сигнала. Почетни регистар чува *x* координату суседног чвора у *EVALUATE* блоку који се тренутно евалуира. Излазни регистар заправо је улазни регистар блок меморије.

Критична путања у суштини представља комбинациону мрежу за рачунање хеуристичке вредности на основу координата. Мултиплексери се користе за одабир позитивног резултата одузимања, што је еквивалентно рачунању апсолутне вредности. Рачун који је реализован овим хардвером описан је формулом за хеуристичку вредност у поглављу 2.1.



Слика 9: Графички приказ критичне путање на RTL нивоу

Разлог зашто је ова путања критична, односно има велико пропагационо кашњење, јесте висок број аритметичких операција везаних у низ. Множења константама не захтевају множаче: множење са 2 реализује се померањем за једно место улево, а множење са 3 померањем и сабирањем. На путањи зато остају два сабирача, један за множење са 3 и један завршни. Ова путања представља уско грло система, у смислу да ограничава максималну учестаност па и брзину рада. Скраћивање ове путање сигурно би повећало максималну учестаност рада система. Могућности скраћивања критичне путање разматране су у поглављу 6.4.

5.2 Поређење брзине рада

Поред разлике у максималној радној учестаности система, постоји и мања разлика од 29 циклуса у броју тактова потребних за извршавање једне итерације.

Извор података	Број тактова по итерацији	Процена времена по итерацији
Процена на системском нивоу	10.007	10.007 × 7,078 ns = 70,83 µs
RTL ниво (тактови из симулације, периода из временске анализе)	10.036	10.036 × 9,9 ns = 99,36 µs

Табела 7: Поређење времена по итерацији процењеног на системском и хардверском нивоу

Однос потребног времена по итерацији процењеног на системском и хардверском нивоу сличан је односу процењених периода јер је број тактова по итерацији скоро па једнак. Остварени број тактова по итерацији износи 100,29 % процењеног броја тактова по итерацији на системском нивоу.

Битно је нагласити да би хардверска имплементација без паралелизације захтевала приближно 4 пута већи број тактова и време од око 397 µs по итерацији.

Главна разлика је у томе што је *HLS* алат проценио перформансе на основу претраге мапе која се обавља проточно, док реализована имплементација приликом претраге обрађује сваки чвор за 4 такта. То значи да се 10.000 чворова претражује, секвенцијално, за 40.000 тактова. Резултат од ~10.000 тактова последица је баш четвороструке паралелизације претраге. Преостали број тактова потребан је за експанзију чвора и прелазак у следећи чвор, али представља мали удео укупног броја тактова па не утиче у великој мери на пропусну моћ.

Коначни закључак поређења јесте да време по итерацији процењено на системском нивоу износи око 71 % времена процењеног за *RTL* реализацију на основу симулације и временске анализе, што се може сматрати успехом. На плочи је такт подешен на 100 MHz, односно на периоду од 10 ns, што је нешто дуже од најкраће периоде од 9,9 ns.

5.3 Поређење искоришћења ресурса

Анализа искоришћења урађена је у *HLS*-у и на хардверском нивоу. Посматрани су *LUT* (енгл. *look-up table*) елементи, *FF* (енгл. *flip-flop*) елементи, блок меморија и *DSP* (енгл. *digital signal processing*) блокови. [6] Резултати су представљени у следећој табели:

Ресурс	*HLS* процена	Језгро после имплементације	Цео систем после имплементације	Доступно на чипу
LUT	1.233 (7,0 %)	563 (3,2 %)	1.588 (9,0 %)	17.600
FF	520 (1,5 %)	333 (0,9 %)	1.186 (3,4 %)	35.200
BRAM (18K)	10 (8,3 %)	0	20 (16,7 %)	120
DSP	2 (2,5 %)	0	0	80

Табела 8: Поређење процењеног и оствареног искоришћења ресурса

Постоје одређена одступања, али су она смислена и могу се објаснити.

Прво одступање је разлика у искоришћености блок меморије између *HLS* процене и језгра у хардверу. Она настаје јер је *HLS* алат меморију третирао као део акцелератора, док је у хардверу инстанцирана одвојено. Зато је смисленије, по критеријуму искоришћења меморије, процену поредити са читавим системом, који користи тачно двоструко више блокова. Разлог је то што је алат за синтезу на високом нивоу уклонио координате родитеља, јер се током рада акцелератора само уписују, а никада не читају. За те податке потребно је 14 бита по чвору (поглавље 3.2), односно укупно 140 kb. Један блок меморије има капацитет од 18 kb, па је за њих потребно најмање 8 блокова. Стварна разлика је 10 блокова, због несавршеног паковања података у меморију.

Друго одступање је то што је *HLS* синтеза искористила два *DSP* блока. Највероватније су искоришћени за множење бројем 100 при рачунању адресе чвора. У хардверу ових блокова нема, јер је *Vivado* множења константом заменио једноставнијим операцијама, уз нешто већи утрошак *LUT* елемената.

Преостале разлике последица су различитог приступа имплементацији. Само језгро заузима више него двоструко мање *LUT* елемената и око трећину мање *FF* елемената од процене, упркос четворострукој паралелизацији, јер је процена дата за аутоматски генерисану архитектуру са проточном обрадом, док је језгро ручно писана машина стања. Читав систем захтева веће искоришћење од процене, јер садржи и блокове које она не обухвата: *AXI4-Lite* омотач, блокове за рутирање меморије и *AXI* инфраструктуру преко које меморији приступа процесор (поглавље 4.7.2).

6. Предлози даљег рада

По завршетку рада, сагледањем читаве слике, истичу се четири правца даље оптимизације и проширења рада.

6.1 Предлог 1: имплементација проточне обраде

По узору на *HLS* имплементацију, предлог за даљу оптимизацију рада био би увођење проточне обраде претраге мапе. Штавише, проточна обрада би се могла упарити са приступом паралелизације који је примењен у раду како би се рад система додатно убрзао. Проточна обрада би обухватила прорачуне *MOVE* блока, узастопно ишчитавајући чворове, поредећи их са тренутним кандидатом за следећи чвор и чувајући адресу чвора са нижом хеуристиком. Тако би се просечан број тактова потребан за обраду једног чвора смањио на вредност веома близу јединице.

6.2 Предлог 2: повећање фактора паралелизације

Начин на који је меморија организована допушта практично произвољан фактор паралелизације, па нема много смисла зауставити се код фактора 4. Једино ограничење при додатној паралелизацији могло би бити повећано пропагационо кашњење комплекснијег кола за декодирање адреса, али благо повећање периоде је врло мала цена за вишеструко убрзање претраге мапе. У наставку налази се табела са апроксимацијама времена по итерацији претраге са повећањем фактора паралелизације. За периоду такта узета је вредност 10 ns, под претпоставком да ће периода такта бити нешто дужа због већег пропагационог кашњења.

Фактор паралелизације претраге мапе	Време по итерацији
8	50,36 μs
32	12,86 μs
64	6,43 μs

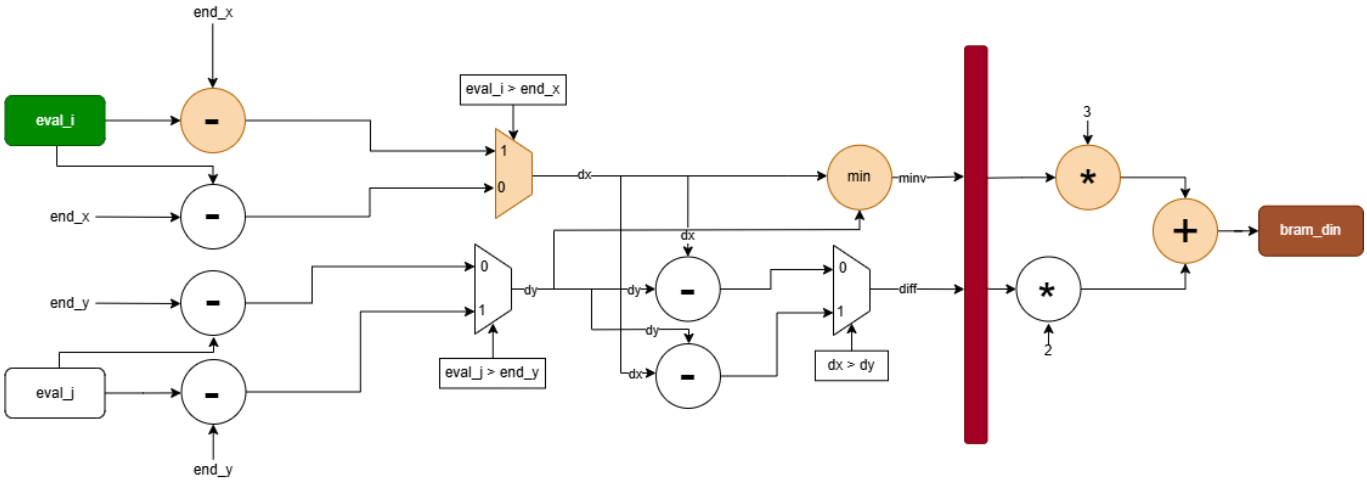
Табела 9: Процене времена по итерацији са повећаним фактором паралелизације

6.3 Предлог 3: увођење кеширања

Од четири предлога даље оптимизације увођење кеширања би било најкомплексније, јер би систем морао да остане отпоран на грешку у случају промашаја. Идеја кеширања у претрази била би чување неког броја, на пример до 20, најбољих чворова (чворови најнижих хеуристика) у регистрима. При експанзији чвора би се хеуристике новооткривених чворова поредиле са кешираним низом и замениле чвор уколико имају нижу вредност хеуристике. Величина кеша морала би се пажљиво бирати. Већи кеш обезбедио би мањи број промашаја, али би и повећао трајање освежавања кеша при свакој експанзији и самим тим повећао време извршавања поготка.

6.4 Предлог 4: скраћивање критичне путање

Увођењем тактованог регистра који „пресеца“ критичну путању она се дели на две мање путање које имају мање пропагационо кашњење. [5] У овом конкретном примеру, најсмисленије било би увести додатни регистар непосредно пре множења константама, која су на шеми приказана као множачи, а реализују се померањем (са 2) и померањем и сабирањем (са 3). Тиме би се рачунање апсолутних вредности и разлике одвојило од два завршна сабирања. Место „пресецања“ на слици означено је усправном црвеном линијом. Додати регистар једноставно би прослеђивао *minv* и *diff* сигнале при растућој ивици такта.



Слика 10: Графички приказ предлога увођења додатног регистра

7. Преглед функционалне верификације и развоја драјвера

Функционална верификација дизајна и имплементација *Linux* драјвера за платформски уређај нису предмет овог рада, али преглед њихових резултата јесте релевантан јер је потврдио исправност имплементације алгоритма, али и открио две грешке у дизајну.

7.1 Функционална верификација

Верификација дизајна урађена је у *SystemVerilog*-у, користећи *UVM* (*енгл. universal verification methodology*) методологију. Коришћен је приступ златних вектора, што значи да су ручно креиране мапе за тестирање циљаних сценарија. Акцелератор и *BRAM* инстанцирани су у окружењу и креиран је један агент који је опонашао рад процесора. Кроз низ тестова утврђено је да акцелератор ради према очекивањима, и да исправно завршава са радом и у случају када је мапа нерешива, то јест не постоји валидна путања од почетног до циљног чвора.

7.2 Развој драјвера и спуштање система на плочу

У последњој фази пројекта, користећи *PetaLinux*, на основу *.XSA* датотеке извезене из *Vivado* алата, генерисани су стабло уређаја и *BOOT.bin* датотека како би се платформски уређај могао препознати из *Linux*-а. Тек у овој последњој фази пронађене су две грешке у хардверу.

Прва грешка била је начин генерисања прекида, који акцелератор шаље процесору по завршетку обраде мапе. У хардверу прекидни сигнал реализован је у виду једнотактног импулса, рачунајући на то да ће процесор препознати растућу ивицу. Процесор није могао да препозна тако кратак импулс.

Проблем је решен повратком на *VHDL* опис, где је у *AXI4-Lite* омотачу имплементиран „лепљиви“ прекид, који остаје на јединици све док га процесор не обрише уписом јединице у бит 0 статусног регистра, који је додат као четврти регистар у мапи акцелератора и поред бита завршетка садржи и битове *busy* и *solved* (поглавље 4.7).

Друга грешка била је мање критична, али није пронађена током верификације па је треба размотрити. Грешка је настала јер је компонента за прилагођавање ширине сигнала прослеђивала погрешан опсег бита адресе меморији. Од 32 бита који пристижу преко *AXI* протокола, прослеђивано је доњих 14 бита. Међутим, 32-битна адреса је адреса бајта а не адреса речи у меморији, па је опсег који се прослеђује померен на бите [15:2] како би се исправно адресирале речи меморије.

Ова грешка је промакла процесу функционалне верификације јер је компонента апстрахована агентом па није била инстанцирана у окружењу, и самим тим грешка се није појавила у тестовима.

7.3 Преглед извршених провера

Преглед свих провера рада система које су извршене дат је у следећој табели:

Врста теста	Број случајева	Очекивани исход	Резултат	Ниво провере	Поновљено након исправки
Решиве мапе, визуелна провера	3 мапе	претрага стиже до циља	3/3	*VHDL* тест бенч, само језгро	не
Упис и читање регистара	2 регистра	прочитано једнако уписаном	испуњено	*UVM*, језгро са омотачем	не
Решиве мапе, поређење целе меморије	10 мапа	свих 10.000 локација једнако референтном моделу	10/10	*UVM*, језгро са омотачем	не
Нерешива мапа	1 мапа	прекид стиже, циљ остаје без родитеља	1/1	*UVM*, језгро са омотачем	не
Узастопна покретања и ресет у току рада	3 узастопна пролаза	исти број тактова, повратак у почетно стање	испуњено	*UVM*, језгро са омотачем	не
Рад читавог система	1	прекид стиже, путања се реконструише	испуњено	плоча, *Linux*	да

Табела 10: Преглед извршених провера

Провере се могу поделити на три нивоа. На првом је *VHDL* тест бенч из поглавља 4.6, којим је на три решиве мапе потврђено да језгро проналази циљ. На другом је *UVM* окружење, које обухвата језгро заједно са *AXI4-Lite* омотачем и чији резултат више не зависи од визуелне процене, јер се садржај свих 10.000 меморијских локација пореди са референтним моделом. На трећем нивоу је рад читавог система на плочи.

Након исправки систем је испитан само на плочи, где је прекид исправно стигао до драјвера, а путања је реконструисана из садржаја меморије. Тиме су „лепљиви“ прекид и исправљено адресирање потврђени у стварном раду.

Из табеле се види и шта није проверено. Адаптер ширине и блокови за рутирање меморије према процесору нису били део ниједне симулације, због чега је друга грешка и откривена тек на плочи.

Јасно је да је верификација још један правац проширења овог пројекта. Предлог даљег рада био би повратак на *UVM* регресије и верификација коначне верзије дизајна, уз увођење рандомизације улазних мапа.

8. Закључак

У оквиру овог рада успешно су спроведени системско пројектовање, архитектурна анализа и хардверска имплементација наменског акцелератора за проналажење путање заснованог на *GBFS* алгоритму на дводимензионалној мрежи димензија 100×100 чворова. Током прве фазе рада израђена је софтверска спецификација у C++ језику, извршено детаљно профајлирање и формирана *Loosely-timed TLM* виртуелна платформа у *SystemC* библиотеци, чиме су благовремено испитани ток података, меморијска организација и хардверско-софтверско партиционисање.

Друга фаза пројекта обухватила је превођење концептуалног модела у конкретан *RTL* опис у *VHDL* језику, уз имплементацију коначног аутомата и четвороструко паралелизованог блока за претрагу меморије. Овако пројектовано IP језгро успешно је интегрисано са Zynq-7000 процесорским системом преко *AXI4-Lite* магистрале на Zybo развојној плочи. Након имплементације у *Vivado* окружењу, функционална верификација коришћењем приступа златних вектора (енгл. golden vectors) и *UVM* методологије показала је на једанаест мапа да акцелератор ради исправно чак и у случају нерешиве мапе. Као завршни корак практичне реализације, успешно су написани одговарајући кернел драјвер за *Linux* оперативни систем и корисничка апликација, након отклањања грешака наведених у поглављу 7.2. Након тих исправки рад система потврђен је на плочи.

Као главни недостатак и простор за даља унапређења истиче се одабрани степен паралелизације претраге. Иако је остварена четворострука паралелизација донела значајно убрзање у односу на непаралелизован дизајн, анализа архитектуре показује да је фактор паралелизације могао лако бити повећан на 8. Тиме би се додатно преполовило време потребно за претрагу мапе по итерацији, па би коначне перформансе реализованог хардвера чак и премашиле почетне процене дефинисане током анализе на системском нивоу. Упркос овом ограничењу, акцелератор представља заокружено решење, проверено симулацијом и испитивањем на плочи, којим је на практичном примеру спроведен *ESL* ток пројектовања, од софтверске спецификације до *RTL* реализације, уз поређење *HLS* процене са оствареним резултатима.

9. Литература

[1] Grötker, T., Liao, S., Martin, G. and Swan, S. (2002), *System Design with SystemC*, ISBN-10: 1402070721, ISBN-13: 9781402070723, Kluwer Academic Publishers, Boston, MA, USA

[2] Ghenassia, F., ed. (2005), *Transaction-Level Modeling with SystemC: TLM Concepts and Applications for Embedded Systems*, ISBN-13: 9780387262321, Springer, New York, NY, USA

[3] Russell, S. and Norvig, P. (2020), *Artificial Intelligence: A Modern Approach*, 4th ed., ISBN-10: 0134610997, ISBN-13: 9780134610993, Pearson, Hoboken, NJ, USA

[4] Zhou, Y., Zhu, X., Zhang, S. and Sun, H. (2020), "FPGA Implementation of A* Algorithm for Real-Time Path Planning", *International Journal of Reconfigurable Computing*, vol. 2020, article ID 8896386, https://onlinelibrary.wiley.com/doi/10.1155/2020/8896386 (датум приступа: 2026-09-26)

[5] Chu, P. P. (2006), *RTL Hardware Design Using VHDL: Coding for Efficiency, Portability, and Scalability*, ISBN-10: 0471720925, ISBN-13: 9780471720928, Wiley-IEEE Press, Hoboken, NJ, USA

[6] AMD (2024), *Vitis High-Level Synthesis User Guide (UG1399)*, https://docs.amd.com/r/en-US/ug1399-vitis-hls (датум приступа: 2026-10-06)

[7] Arm (2021), *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022, верзија H.c, https://developer.arm.com/documentation/ihi0022/hc (датум приступа: 2026-10-06)